



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

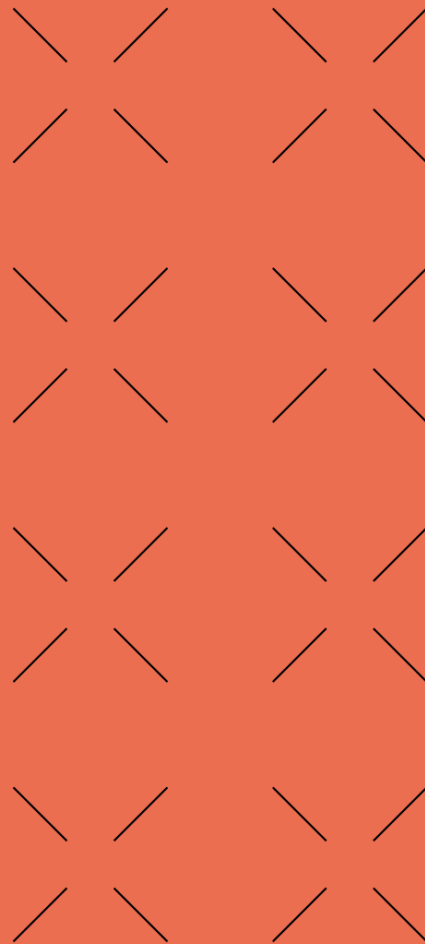
CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 05 — Funcions i mètodes

Bloc 02

1r CFGS DAW · Programació

Sergi García Barea



Què veurem en este bloc



Àmbit de variables

Cada mètode és una casa



Errors freqüents

Els 7 entrebancs i el diagnòstic



Divideix el problema

main com a director d'orquestra



Be the Code

Tallar un programa amb les
teues mans



Repàs general

Qui soc? · decisions ·
entrevistes



Un avançament

La cadena de crides (recursió,
U08)

Àmbit de variables

La idea en una frase

“

L'àmbit (*scope*) és el tros de codi on una variable **existeix**: dins d'un mètode només viuen els seus paràmetres i els seus locals, i **el que naix ahí, mor ahí**.



Quan veges `cannot find symbol` en una variable, gairebé sempre és una de dos: l'has escrit malament (*typo*) o estàs usant-la **fora de la seua casa**.

Cada mètode és una casa

Casas.java

Java

```
public class Casas {  
    public static void sumar() {  
        int total = 10;  
        total += 5;  
        System.out.println(total);    // 15  
    }  
    public static void restar() {  
        int total = 100;  
        total -= 5;  
        System.out.println(total);    // 95  
    }  
    public static void main(String[] args) {  
        sumar();  
        restar();  
        // System.out.println(total); → ¡error!  
    }  
}
```

```
$ java Casas
```

```
15
```

```
95
```

```
error: cannot find symbol: variable total
```

i Dos mètodes amb una variable `total`

ahora: zero problemes. Són **dos totals diferents**, una en cada casa.

! `main` no pot usar `total` : **no existeix** fora de `sumar()`. El mètode acabà, la seua pila es desfet i `total` ja no està.

El mapa de les cases

L'àmbit: el que naix en una casa, mor en la casa

main

```
System.out.println(total);  
✗ cannot find symbol
```

sumar()

```
int total = 10;  
total += 5; → 15
```

mor ací 💀

restar()

```
int total = 100;  
total -= 5; → 95
```

mor ací 💀

✓ Dos total ahora: cap problema (són dos variables diferents).
Entre cases només viatja el que entra (arguments) i el que ix (return)

Les regles de l'àmbit

- 1 **Paràmetres i locals** es veuen només **dins del seu mètode** (de la firma a la clau de tancament)
- 2 **Un bloc també és una casa menuda:** la `i` d'un `for` no existeix fora de les seues claus (U04)
- 3 **L'ordre importa dins del mètode:** només pots usar una variable **després** de declarar-la
- 4 **Dues germanes no es barallen:** repetir noms entre mètodes és normal i sa

i En acabar el mètode, les seues variables **moren**: només sobreviu el que tornes amb `return` o el que imprimixes.

El que entra per la porta també és de la casa

Triplicar.java

Java

```
public static int triplicar(int numero) {  
    int resultado = numero * 3;  
    return resultado;  
}  
  
// en main:  
int puntos = 10;  
triplicar(puntos);  
System.out.println(puntos); // 10
```

⚠️ `triplicar` va treballar amb la seua còpia de `puntos`: el teu mètode pot usar el valor, però **no reescriure la variable de qui crida**.

✅ Per a «tornar» canvis: el mètode retorna el nou valor amb `return` i qui crida el guarda.

i Els mètodes només comparteixen el que es passen (arguments) o el que es torna (`return`).
No hi ha ventanetes laterals.

Exercici: existeix esta variable?

Alcance.java

Java

```
public class Alcance {  
    public static void a() {  
        int x = 1;  
    }  
    public static void b() {  
        System.out.println(x);  
    }  
    public static void main(String[] args) {  
        b();  
    }  
}
```



Què passa? (A) imprimeix 1 · (B) imprimeix 0 · (C) no compila · (D) excepció en executar



Solució: la C. `x` viu i mor dins de `a()`: el mètode `b()` no la coneix. És un `cannot find symbol` de **compilació**: ni tan sols arriba a executar-se. Les variables no viatgen entre mètodes per art d'engany.

Mini-chequeig: l'àmbit



Respon en 30 segons:

1. Poden dos mètodes tindre una variable local `contador` ahora?
2. Existeix la `i` del `for` fora de les seues claus?
3. Sobreui `resultado` a l'acabar el mètode que la declara?
4. El meu mètode no canvia el saldo que li he passat. És un bug?



Respostes:

- Sí: cadascuna viu en el seu mètode, són **dos variables diferents**
- No: el seu àmbit acaba en la clau de tancament del `for`
- No: les locals **moren** en eixir; només sobreui el que tornes
- No és bug: els arguments primitius arriben **copiats**; si vols el resultat, `return`

Errors freqüents

La idea en una frase

“

Els errors amb mètodes **no són aleatoris**: tenen nom, missatge i arregla. Apren els set i el 90% de les teues hores de frustració s'evaporen.



Els sis primers els veu **CONRAD** (el compilador) en roig abans d'executar. El setè és d'execució: **el detecta el teu cervell** llegint noms en veu alta.

La galeria dels entrebancs (1-4)

#	Codi sospitós	Error de CONRAD	Arregla
1	<code>int</code> amb un camí sense <code>return</code>	<code>missing return statement</code>	Tanca tots els camins amb <code>return</code>
2	<code>pintar(3, "roig")</code> quan la firma és <code>(String, int)</code>	<code>incompatible types</code>	Respecta l' ordre i el tipus de la firma
3	<code>suma(1, 2, 3)</code> quan la firma demana 2	<code>wrong number of arguments</code>	La firma mana: 2 paràmetres = 2 arguments
4	<code>int r = media(3, 4);</code> amb <code>media</code> void	<code>void cannot be converted</code>	Que el mètode faci <code>return</code> ; imprimir és de qui crida

 El 4 és el **pecat més comú de la unitat**: imprimir quan has de tornar. El valor s'imprimix i desapareix per a tothom.

La galeria dels entrebancs (5-7)

#	Codi sospitós	Error	Arregla
5	<code>void saludar() {...}</code> cridat des de <code>main</code>	<code>non-static method ... cannot be referenced from a static context</code>	De moment, els teus mètodes porten static
6	<code>saludar</code> (sense <code>()</code> i <code>;</code>)	no és una crida	La crida és <code>saludar();</code>
7	<code>mostrarTotal();</code> quan volies <code>mostrarMedia()</code>	cap: compila i executa... i ment	Llig el nom de cada crida en veu alta

 **El 7 és el més perillós dels set:** sense error de compilació, executa i ment. D'ací a dos setmanes, el que va escriure eixe codi serà un desconegut (tu mateix).

Com diagnosticar sense perdre't

Diagnòstic d'errors amb mètodes: 3 preguntes, en ordre

1 · ¿Compila?

sí

2 · ¿Fa el que esperaves?

sí

3 · ¿Imprimeix algo rar?

no

no

sí

llegix el missatge:
1a línia → QUE
2a línia → ON

error 7: has cridat al mètode
correcte? error 2: ordre i tipus
dels arguments correctes?

traça la crida: qui imprimeix
i qui torna? uses la còpia
que arriba per paràmetre?

cannot find symbol · wrong number
of arguments · incompatible types ·
missing return statement

⚠ No pegues l'error al buscador abans de llegir-lo sencer: la 1a línia diu què, la 2a on

Exercici: el taller d'arregles

Mayor.java

Java

```
// hauria de tornar el major de dos
// nombres, però té 2 errors:
public static int mayor(int a, int b) {
    if (a > b) {
        return a
    }
    return 0;
}
```



Un error de compilació i un de lògica.
Quins són?



Solució:

- **Compilació:** falta `;` després de `return a`
- **Lògica:** el segon camí torna `0` en lloc de `b`: quan `a ≤ b`, el major és `b`



👉 **Don Tip:** quan tot compile i el resultat no quadre, imprimeix (o millor: **torna**) els valors a mig camí. Depurar és mirar, no endevinar.

Mini-chequeig: errors



Respon sense mirar:

1. Quin error dona `int r = media(3, 4);` si `media` és void i només imprimeix?
2. Quins missatges busques primer en no compilar una crida?
3. Per què l'error 7 no el detecta el compilador?



Respostes:

- `void cannot be converted to int` : res de `void` es pot guardar en una variable
- `cannot find symbol` · `wrong number of arguments` · `incompatible types` · `missing return statement`
- Perquè el nom i els arguments són correctes **sintàcticament**: només tu saps que volies l'altre mètode

Divideix el problema

La idea en una frase

“

Dividir és escriure un `main` que es llija com un **resum** («llegir dades, calcular mitjana, mostrar informe») i amagar cada frase **en un mètode amb el seu nom**.



El director d'orquestra **no toca els instruments**: coordina. Un `main` de 40 línies barrejant lectura, càlcul i format és un full de receptes, no un índex del programa.

El director i la seua orquestra

InformeNotas.java

Java

```
public static void main(String[] args) {  
    double[] notas = leerNotas(5);  
    double media = calcularMedia(notas);  
    boolean suspensa = tieneSuspensa(notas);  
    mostrarVeredicto(media, suspensa);  
}
```

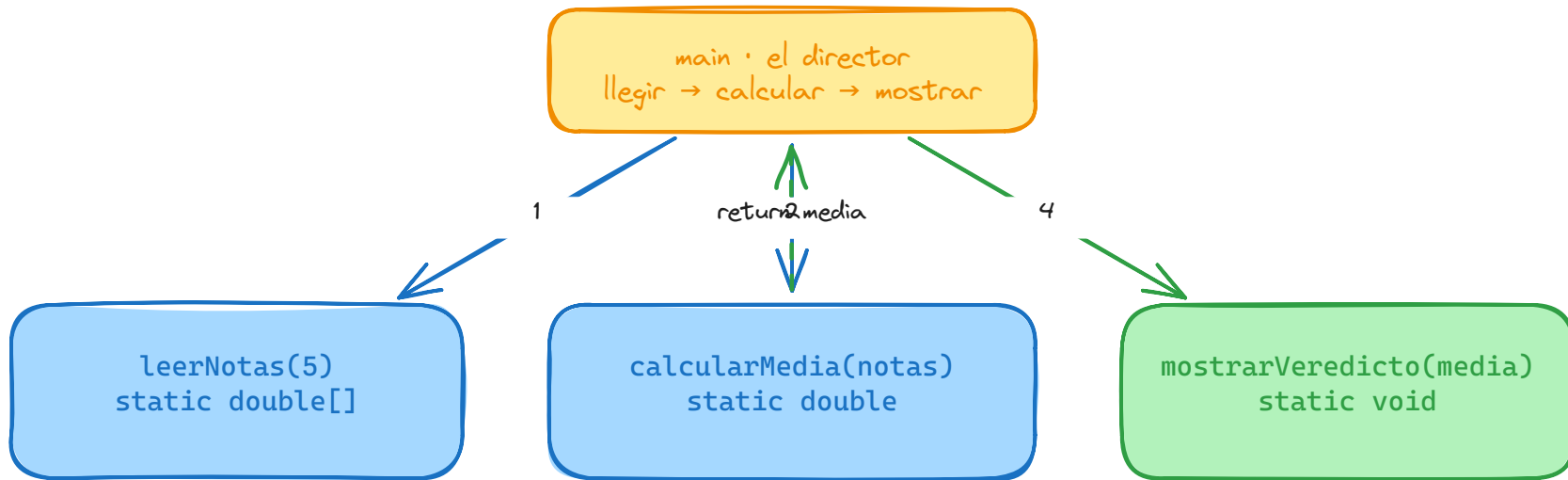
Quatre línies, quatre verbs, **zero aritmètica**.

El mètode dels verbs: cada verb amb parèntesis en un comentari és un candidat a mètode.

- 1 Escriu el **comentari-objectiu** («llegir les notes»)
- 2 Crea el **mètode** amb eixe nom: paràmetres si necessita dades, retorn si torna algo
- 3 **Mou el codi** al cos i deixa la crida en `main`
- 4 **Executa:** si fa el mateix que abans, has trossejat bé. Un mètode cada volta

main com a índex del programa

Divideix el problema: el director no toca els instruments



Cada verb amb nom propi = un mètode · una responsabilitat per mètode
calcularMediaYMostrarYGuardar són tres mètodes disfressats d'un

On està el límit?

Es queda en main	Va al seu mètode
La seqüència de passos	Un pas amb nom propi
Un <code>if</code> de dues línies de flux	Un bloc que repeteixes o que «fa una cosa»
La crida final d'impressió	El compte, la lectura, la cerca, el format

i No hi ha llei de «més de N línies»: la brúixola és una responsabilitat per mètode.

 `calcularMediaYMostrarYGuardar` són **tres mètodes disfressats d'un**: si el nom necessita una «i», són dos mètodes.

  Si necessites un comentari per explicar un bloc de 5 línies, eixe bloc **ja té nom**: fes-lo mètode i que el nom parle per ell.

i  **Don Tip**: escriu la crida que **volgues tindre** (`int max = encontrarMaximo(notas);`) i després fes que existisca. El codi es dissenya cap avant.

Exercici: el verb amagat

Positius.java

Java

```
public static void main(String[] args) {  
    int[] datos = { 3, 7, 2 };  
    boolean todoPositivo = true;  
    for (int i = 0; i < datos.length; i++) {  
        if (datos[i] ≤ 0) {  
            todoPositivo = false;  
        }  
    }  
    System.out.println("¿Tot positiu? "  
        + todoPositivo);  
}
```



Quin mètode (nom i firma) treguries d'eixe main?



Solució: el verb és «tots positius» → `todosPositivos` .

```
static boolean todosPositivos(int[] datos) {  
    for (int i = 0; i < datos.length; i++) {  
        if (datos[i] ≤ 0) { return false; }  
    }  
    return true;  
}
```

Necessita dades? Sí → `int[] datos` .

Torna un judici? Sí → `boolean` .

Mini-chequeig: dividir



Respon en 30 segons:

1. Quines tres preguntes et fas en dissenyar un mètode?
2. Quin és l'ofici de `main` en un programa ben trossejat?
3. Quants blocs trossegues d'una volta?



Respostes:

- Com es diu (**quin verb fa**)? Quines dades necessita (**paràmetres**)? Què torna (**retorn o void**)?
- **Coordinar**: llegir → calcular → mostrar, en crides llegibles
- **Un sol**, executant després de cada extracció: si algo es trenca, saps què va ser

Repàs: Be the Code

La idea en una frase

“

Este punt no té teoria nova: té un programa de 30 línies que has de **trossejar amb les teues mans**. Si ho fas, domines la unitat sencera.



La missió: convertir l' `InformeNotas` en una **orquestra de mètodes** sense canviar el que imprimeix. Abans d'escriure: **quants verbs** veus? Quines dades necessita cada un i **què torna**?

El punt de partida (tot en main)

InformeNotas.java

Java

```
public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);
    double suma = 0;
    boolean haySuspensa = false;
    for (int i = 0; i < 5; i++) {
        System.out.print("Nota " + (i + 1) + ": ");
        double nota = teclado.nextDouble();
        suma += nota;
        if (nota < 5) { haySuspensa = true; }
    }
    double media = suma / 5;
    System.out.println("Mitjana: " + media);
    if (haySuspensa) { System.out.println("Hi ha suspensa: a per ella."); }
    else if (media ≥ 7) { System.out.println("¡Qué nota!"); }
    else { System.out.println("Aprovat amb genolls."); }
}
```

i Pista: **tres verbs** amb nom propi: llegir → calcular → mostrar. I una pregunta (¿hi ha suspensa?) que també vol ser mètode.

Pas a pas (pistes que no regalen el final)

- 1 `static double[] leerNotas(int cantidad)` — llegir, amb Scanner dins
- 2 `static double calcularMedia(double[] notas)` — calcular, torna la mitjana
- 3 `static boolean tieneSuspensa(double[] notas)` — pregunta, torna `true/false`
- 4 `static void mostrarVeredicto(double media, boolean suspensa)` — contar, `void`
- 5 Reescriu `main` com a resum: **llegir** → **mitjana** → **suspensa** → **mostrar**, sense un sol `for`



L'eixida ha de ser **idèntica** abans i després. Si canvia, has mogut una línia de més: **refactoritzar és canviar la forma, mai el comportament.**

El Lío: el refactor malogràt

MalRefactor.java

Java

```
public static void main(String[] args) {  
    double m = media(3, 4, 5);  
    System.out.println(m);  
}  
  
static void media(int a, int b, int c) {  
    double resultado = (a + b + c) / 3.0;  
    System.out.println(resultado);  
}
```

 **No compila:** `main` intenta guardar en `double m` el resultat d'un mètode `void` → `void` cannot be converted to double.

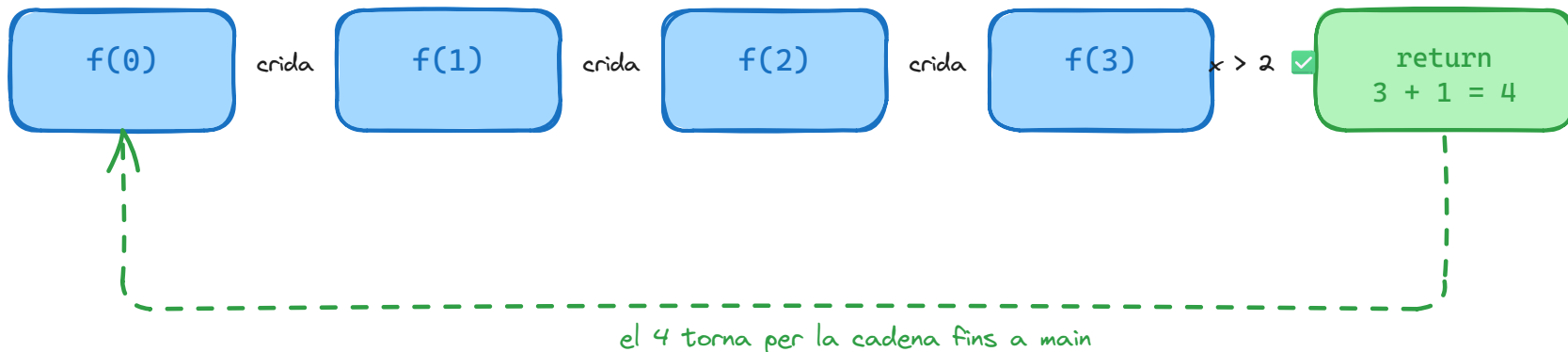
 **L'arregle correcte:** `static double media(int a, int b, int c) { return (a + b + c) / 3.0; }` — i fora el `println` de dins.

 **Calcula → torna. Decideix → imprimeix.** Si el mètode es diu `media`, el seu ofici és calcular, no contar.

La cadena de crides (avançament de la U08)

Repàs: una cadena de crides → el valor torna per on va vindre

```
static int f(int x) {  
  if (x > 2) return x + 1;  
  return f(x + 1);  
}
```



⚠ Un mètode que es crida a si mateix = recursió (U08).
Si no arriba mai al seu cas base, la pila s'acaba i explota *

Repàs: qui sóc?



Endevina el concepte:

1. Sóc la variable que un mètode rep en ser cridat, declarada en la seua firma.
2. Sóc el valor que arriba en la crida, en l'exacte lloc on polses el gatell.
3. Sóc la regió on una variable existeix: de la firma a la clau de tancament.
4. Sóc el final del mètode: tanque l'execució i entregue el valor promés.
5. Sóc un mètode que torna `true` o `false` : `esPar` , `tieneSuspensa` .



Respostes: 1 → **el paràmetre** · 2 → **l'argument** · 3 → **l'àmbit (scope)** · 4 → `return` · 5 → **el predicat**

El joc de les decisions

1. Quina és la diferència real entre paràmetre i argument?
2. Què imprimeix `println(truco(7))` ?
3. Poden `a()` i `b()` tindre totes dues una variable `x` ?
4. Quin error dona `int r = suma(1, 2);` si la firma demana 3?
5. Quan uses `void` ?



Solucions:

- **1-b:** declaració vs crida
- **2-a:** 10 (cau en `x > 5 : 7 + 3`)
- **3-b:** sí, són cases distintes
- **4-a:** `wrong number of arguments`
- **5-a:** només efecte, sense valor a entregar

Preguntes d'entrevista (Java junior)

- 1 Explica'm, com si jo fora ta àvia, la diferència entre **paràmetre** i **argument**
- 2 Escriu un mètode que torne `true` si tots els nombres són positius. **Per què boolean i no un println?**
- 3 Què és l'**àmbit** d'una variable i per què existeix?
- 4 El teu mètode compila però el resultat **no arriba a main**. Per què pot ser?
- 5 Quan faries un mètode `void` i quan amb retorn? Un exemple de cada
- 6 Refactoritza un `main` de 40 línies: **per on comences** i com proves que no l'has trencat?

✓ 💡 Per a la 2: **boolean és reutilitzable** (comparar, combinar, tornar a usar); un `println` només es veu una volta i desapareix.

Resum del bloc 02



Àmbit

El que naix en una casa, mor en la casa; entre mètodes: arguments i return



7 errors

6 els veu CONRAD; el 7 (mètode equivocant) el veus tu



Divideix

main = resum de verbs; una responsabilitat per mètode



La lliçó que ho cobreix tot: **void per a efectes, return per a valors**. Qui calcula torna; qui decideix imprimeix.

